

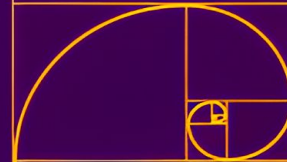
Splay Trees

CSCI 33500: Software Design and Analysis III

Sadab Hafiz

sh3646@hunter.cuny.edu

07



Motivation

An Alternative Approach

→ Benefits of balanced BST

- ◆ Guarantees **logarithmic height**
- ◆ Ensures all operations are **$O(\log(n))$**
- ◆ Example:
 - AVL Trees
 - Red-black Trees

Let's consider the following use case:

- ### →
- Data that has been recently accessed is more likely to be accessed again in the near future

Motivation

An Alternative Approach

Why not AVL Trees in this case?

→ Benefits of balanced BST

- ◆ Guarantees **logarithmic height**
- ◆ Ensures all operations are **$O(\log(n))$**
- ◆ Example:
 - AVL Trees
 - Red-black Trees

Let's consider the following use case:

→ Data that has been recently accessed is more likely to be accessed again in the near future

Splay Trees

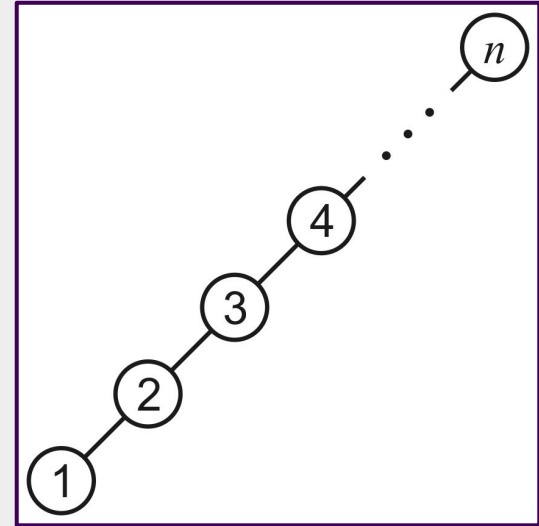
Introduction

- Accessed nodes could be rotated or *splayed* to the root of the tree:
 - ◆ Accessed nodes are splayed to the root during the count/find operations
 - ◆ Inserted nodes are inserted normally and then splayed to the root
 - ◆ The parent of a removed node is also splayed to the root
- Invented in 1985 by Daniel Dominic Sleator and Robert Endre Tarjan

Splay Trees

Insertion at the Root

- It is clear that simply inserting at the root leads to an access time of $O(n)$
- For instance, insert the values $1, 2, 3, 4, \dots, n$ in that order:

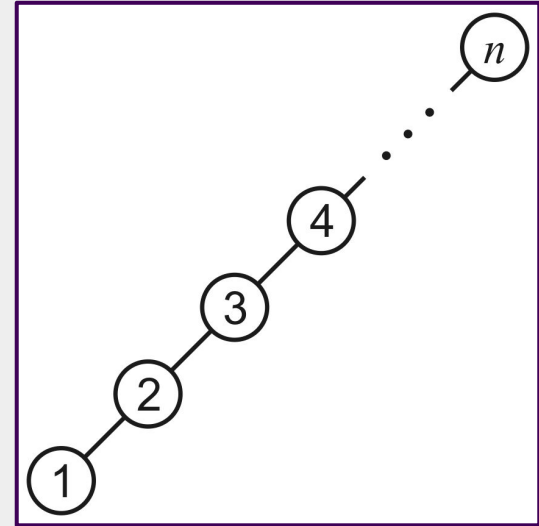


- Now, any access to 1 requires that a linked list be traversed

Splay Trees

Insertion at the Root

- However, we are interested in amortized runtimes
- We only require that n accesses must have $\Theta(n \cdot \log(n))$ runtime

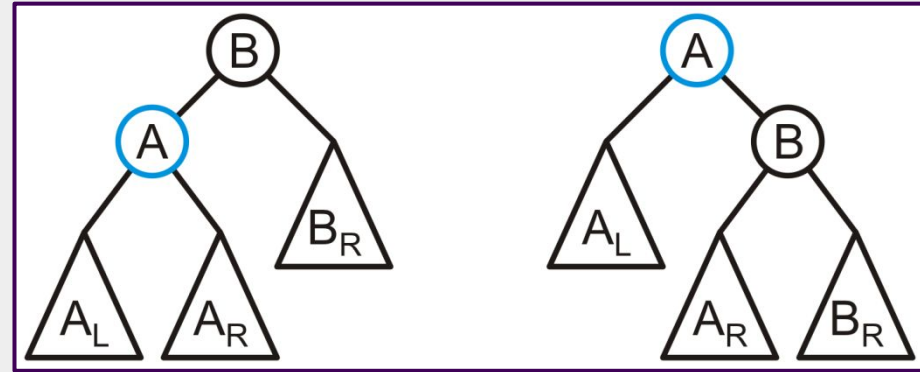


- Thus, $O(\log(n))$ of those accesses could still be $O(n)$

Single Rotations

Move Access Node to the Root

- Before we consider insertions, how can we simply move an access node to the root?
- Consider AVL rotations, for example:

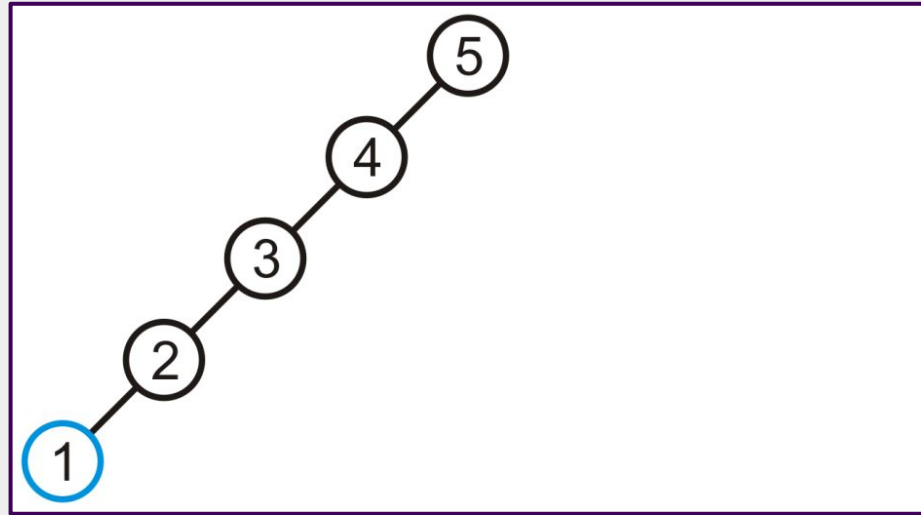


- The above is the simplest rotation
- Unfortunately, as we will see, using just single rotations **does not work**

Single Rotations

Example

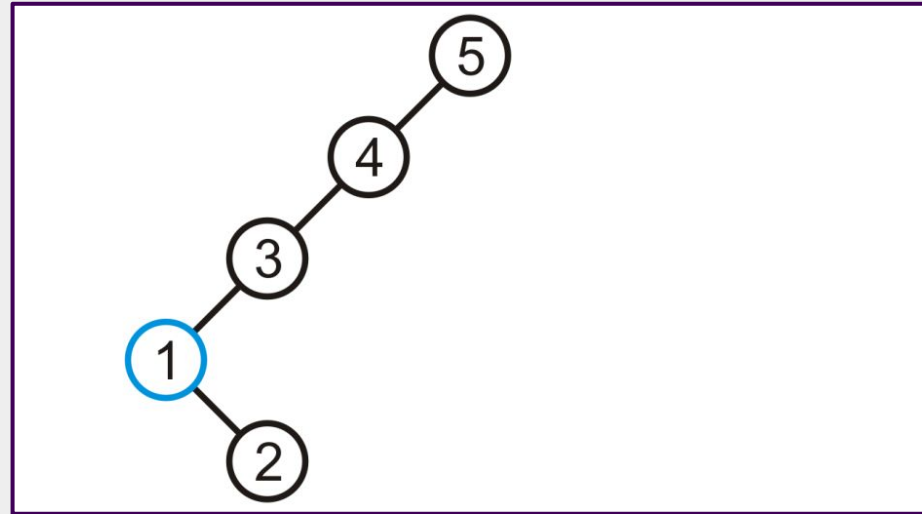
- Consider this splay tree:
- ◆ Contains five entries
 - ◆ Insertion order: 1, 2, 3, 4, and 5
 - ◆ Let us access 1 by finding it and then rotating it back to the root



Single Rotations

Example

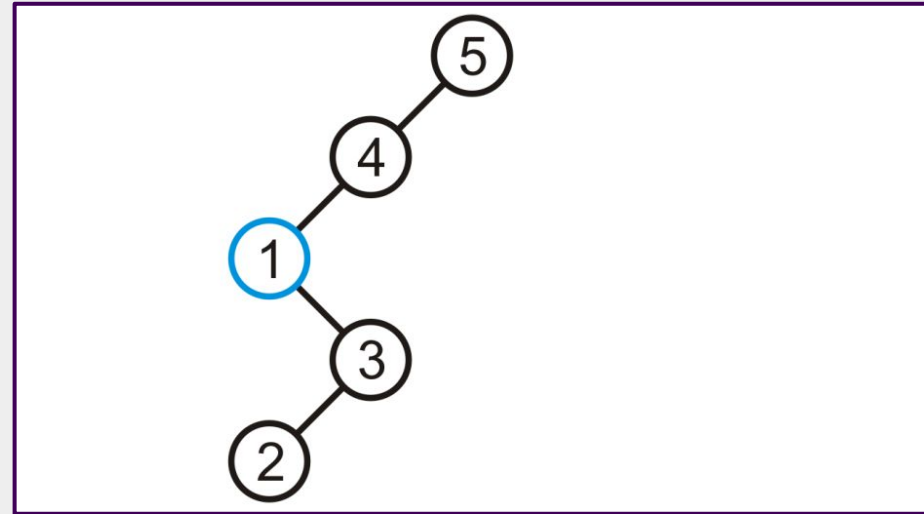
Rotating 1 and 2



Single Rotations

Example

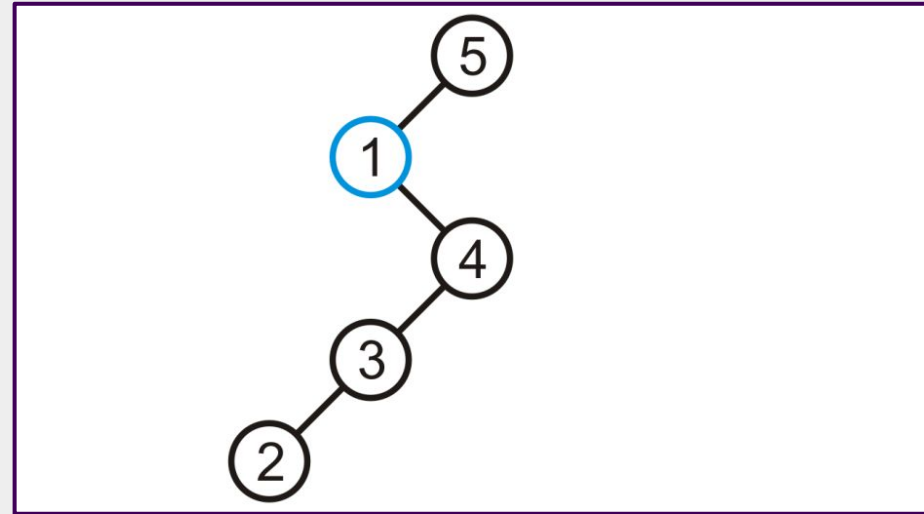
Rotating 1 and 3



Single Rotations

Example

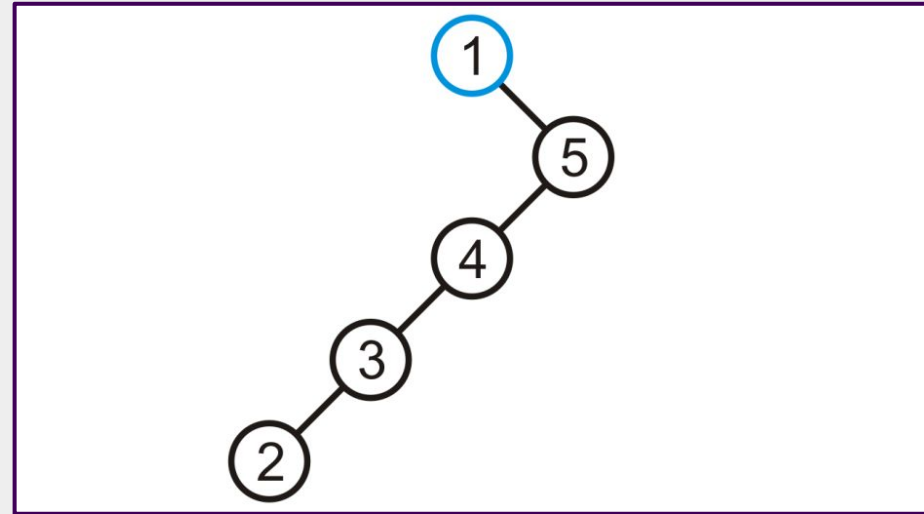
Rotating 1 and 4



Single Rotations

Example

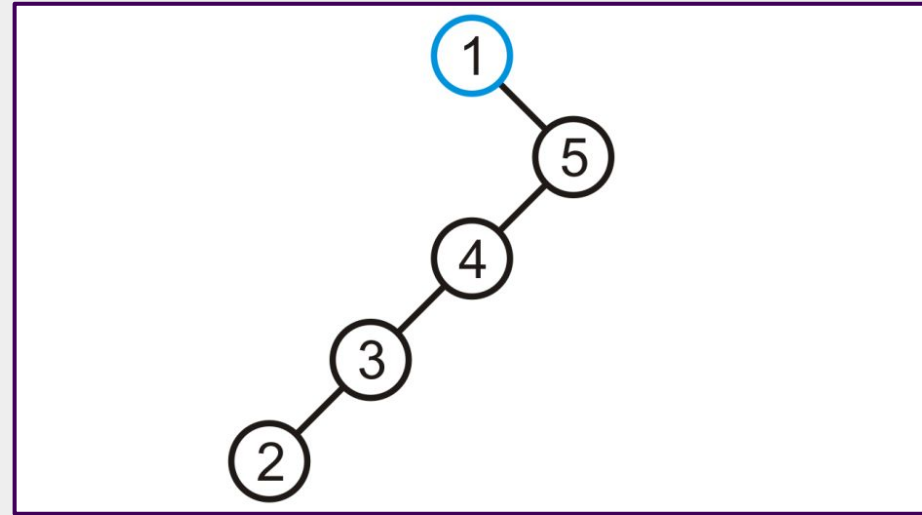
Rotating 1 and 5



Single Rotations

Example

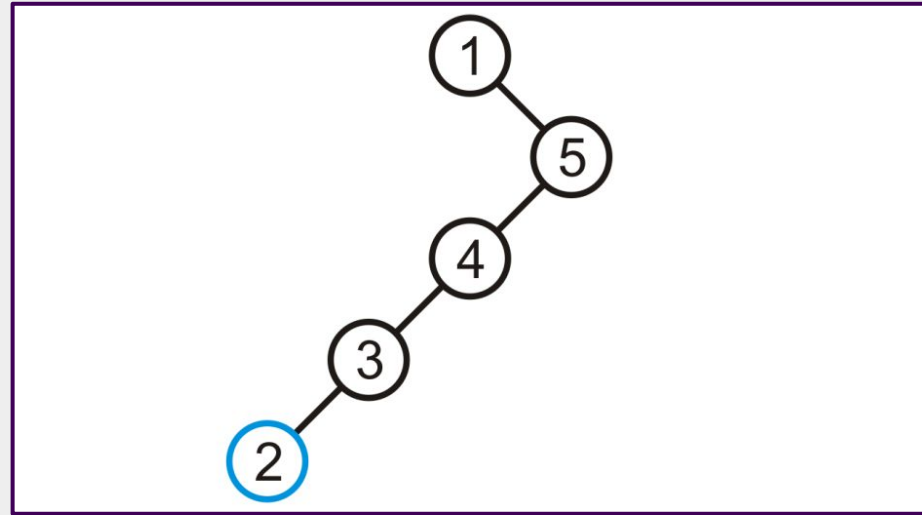
The result still looks like
a **linked list**



Single Rotations

Example

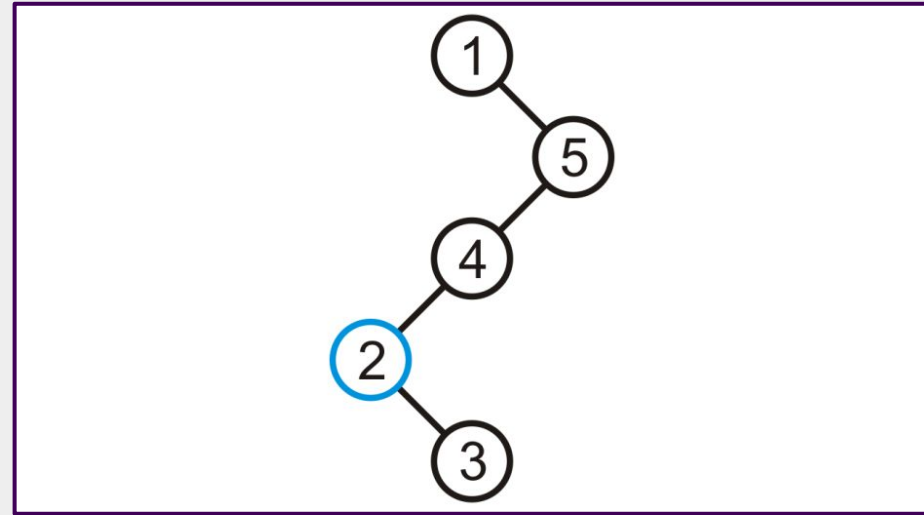
Accessing 2 next **doesn't**
do much either



Single Rotations

Example

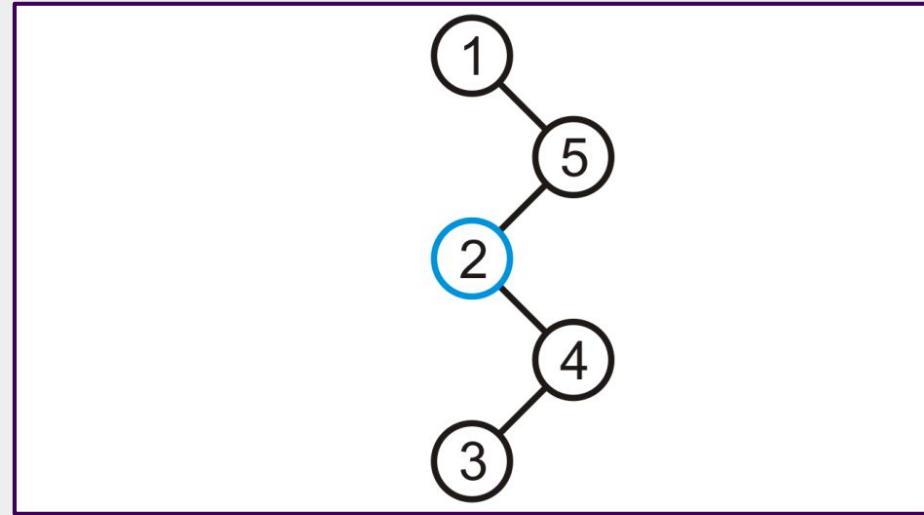
Accessing 2 next **doesn't**
do much either



Single Rotations

Example

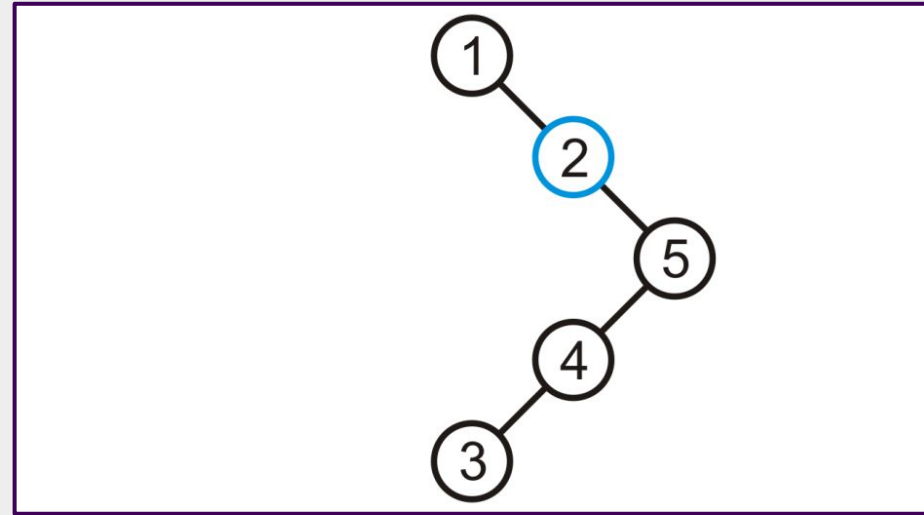
Accessing 2 next **doesn't**
do much either



Single Rotations

Example

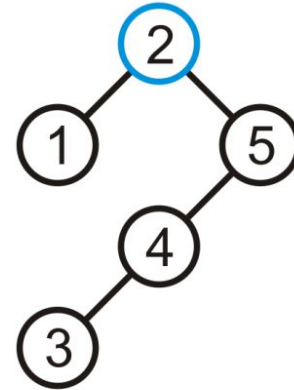
Accessing 2 next **doesn't**
do much either



Single Rotations

Example

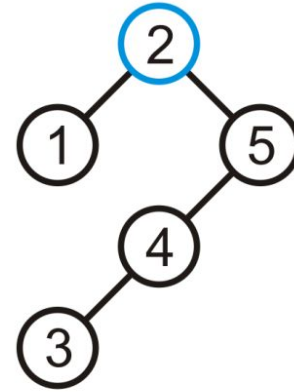
Accessing 2 next **doesn't**
do much either



Single Rotations

Example

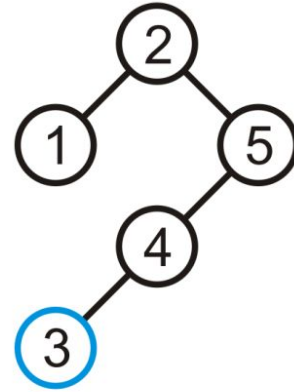
The resulting tree is shallower by **only 1**



Single Rotations

Example

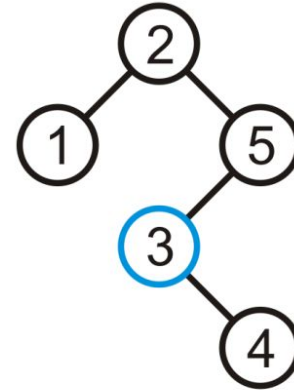
Accessing 3 is **not**
significant either



Single Rotations

Example

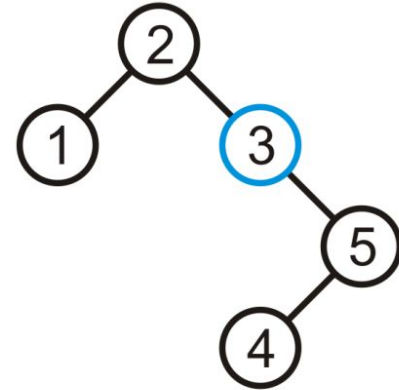
Accessing 3 is **not**
significant either



Single Rotations

Example

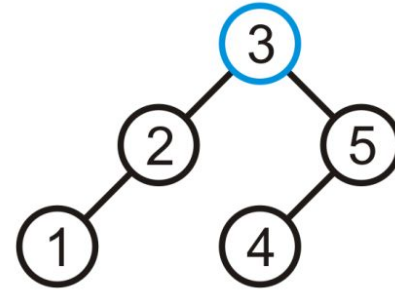
Accessing 3 is **not**
significant either



Single Rotations

Example

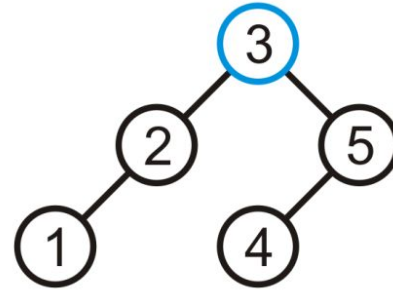
Accessing 3 is **not**
significant either



Single Rotations

Does Not Work

- Essentially, it is two linked lists and the left subtree is turning into the original linked list we started from



Single Rotations

Does Not Work

In a general splay tree created in order:
 $1, 2, 3, 4, \dots, n$

Accessing repeatedly in the order:
 $1, 2, 3, 4, \dots, n$

Requires:

$$\sum_{k=1}^n (n-k) = n^2 - \sum_{k=1}^n k = n^2 - \frac{n(n+1)}{2} = \frac{n(n-1)}{2} = O(n^2)$$

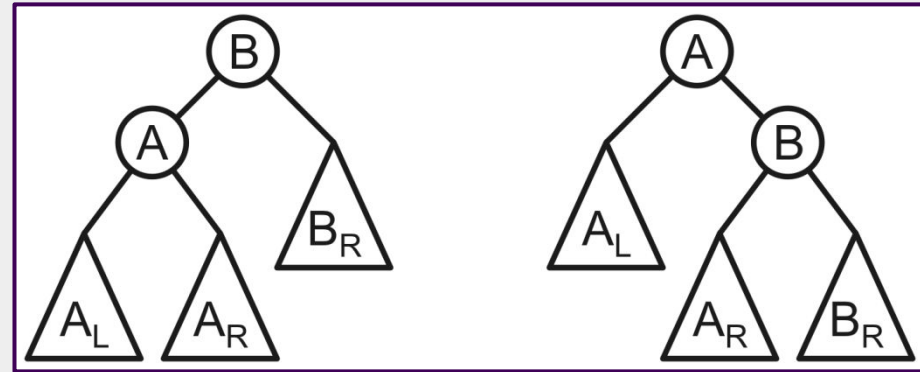
comparisons for n accesses. Thus:

an amortized runtime of $O(n)$

Single Rotations

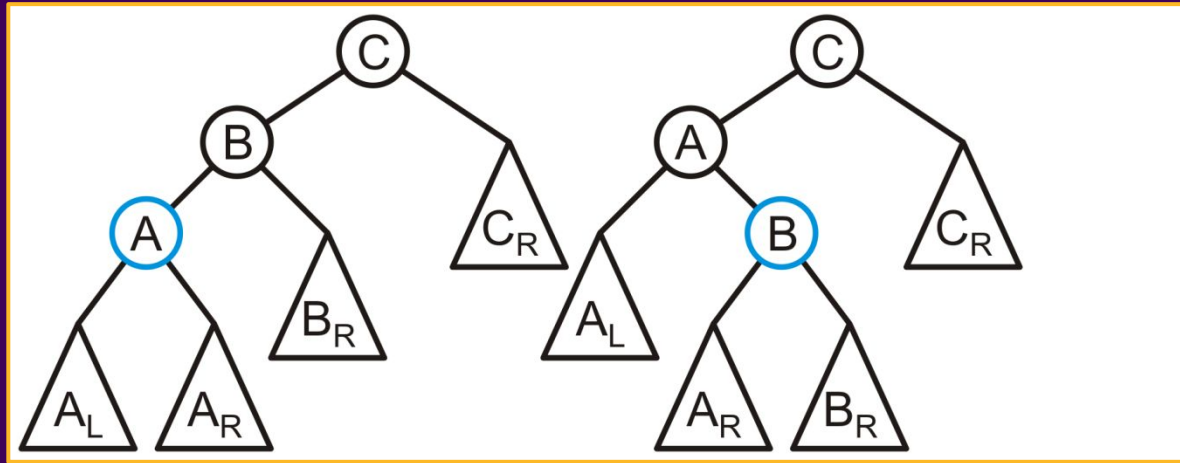
Does Not Work

- Thus, a single rotation will not do
- As demonstrated, it can convert a linked list into a linked list



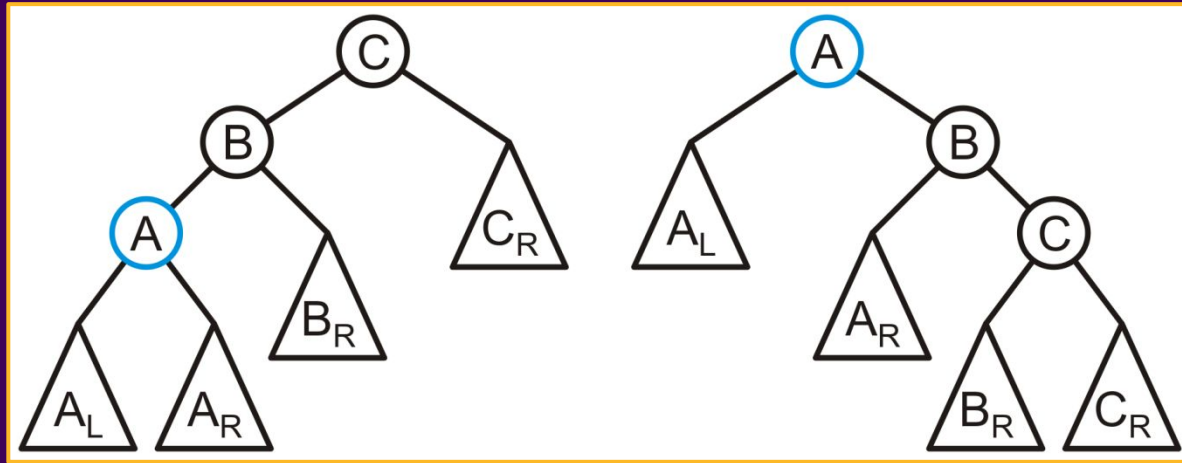
Depth-2 Rotations

- Let's try rotations with entries at depth 2 of the tree
- Suppose we are accessing **A** on the left and **B** on the right



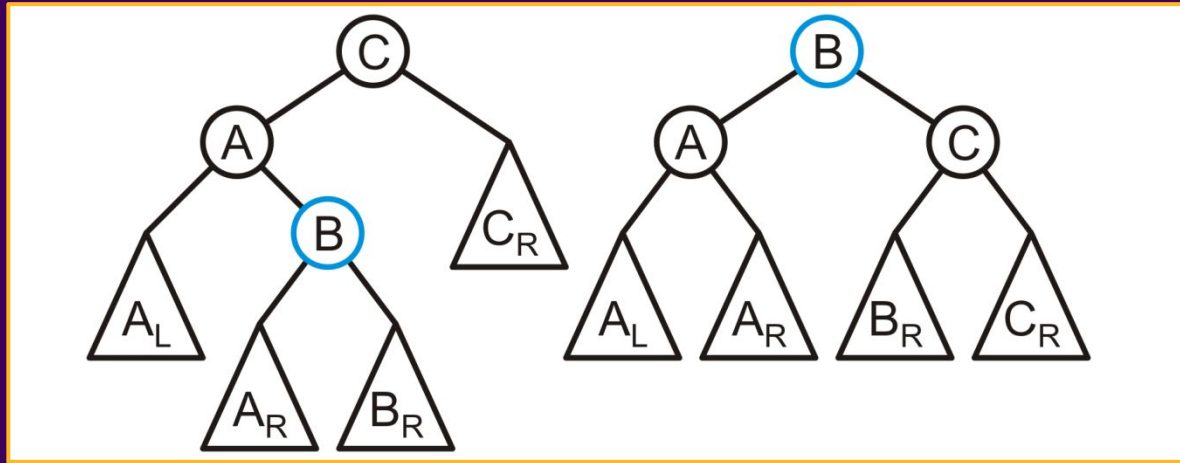
Depth-2 Rotations: Zig-Zig

- In the first case, two rotations at the root bring A to the root
- We call this a *zig-zig* rotation



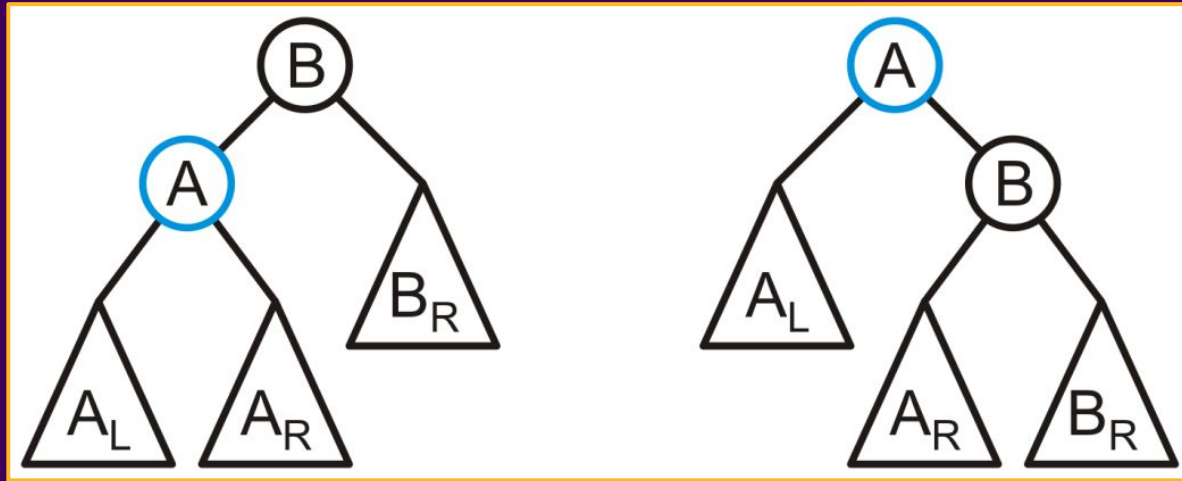
Depth-2 Rotations: Zig-Zag

- In the second case, two rotations bring **B** to the root
- We call this a *zig-zag* rotation



Depth-2 Rotations: Zig

- If the accessed node is a child of the root, we must use a single rotation
- We call this a *zig* rotation



Operations

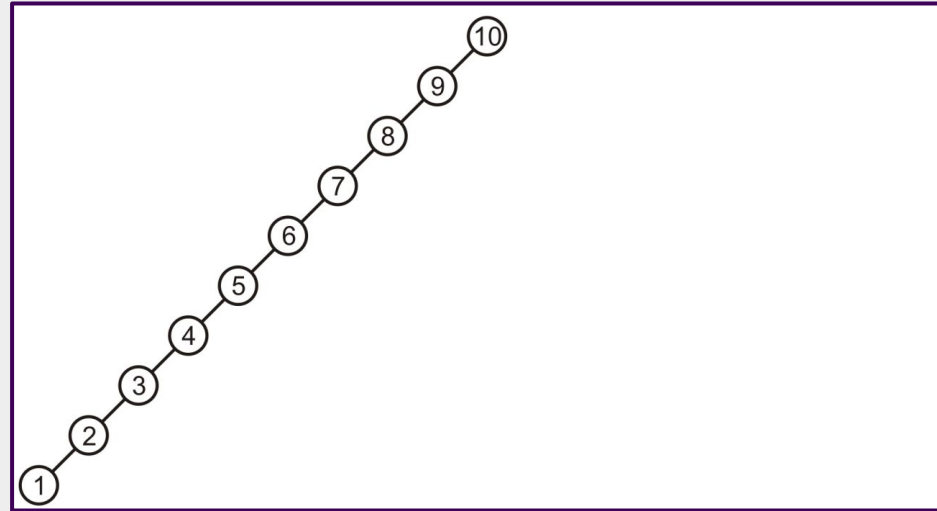
When To Splay?

- **Accessing** any node splays the node to the root of the binary search tree
- **Inserting** a new element into a splay tree follows the BST model:
 - ◆ Insert the node as per a standard BST
 - ◆ Splay the object to the root
- **Removing** a node also follows the pattern of a BST model:
 - ◆ Regular BST removal (three cases)
 - ◆ Splay the parent of the removed node to the root of the BST if tree is not empty
 - ◆ Alternatively, splay the node being removed to the root followed by standard BST removal of that node

Example

Splay Tree

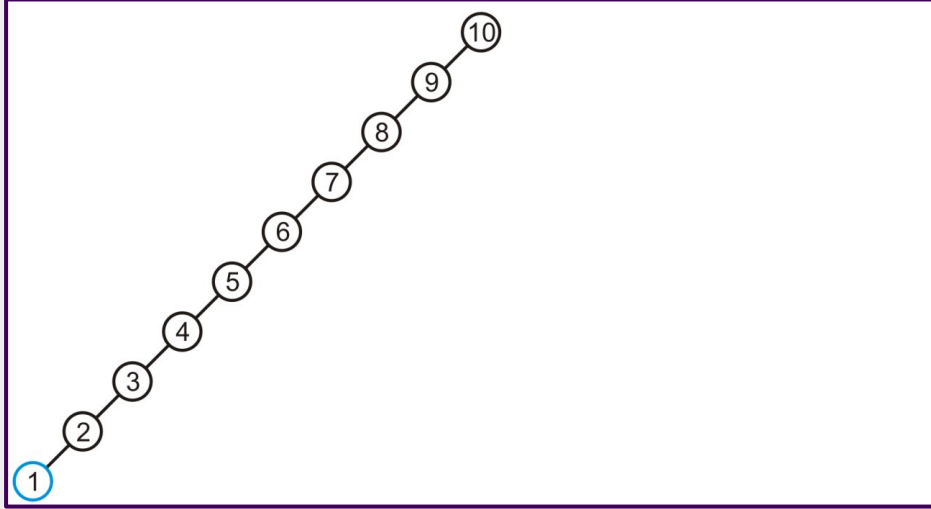
→ Inserting 1 through 10, in that order, will produce the splay tree:



Example

Splay Tree

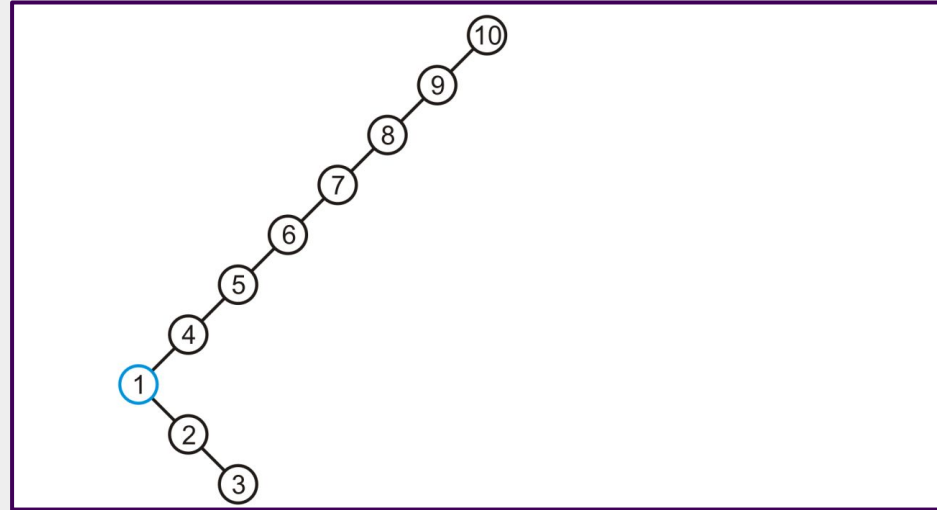
- We will repeatedly access the **deepest node** in the tree
 - ◆ With each operation, this node will be splayed to the root of the tree
- We begin with a zig-zig rotation



Example

Splay Tree

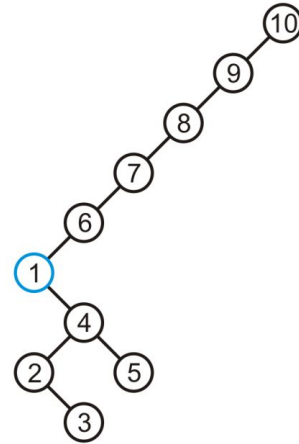
→ Followed by another zig-zig rotation



Example

Splay Tree

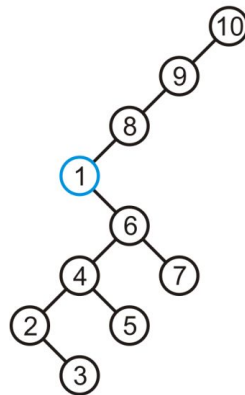
→ And another zig-zig rotation



Example

Splay Tree

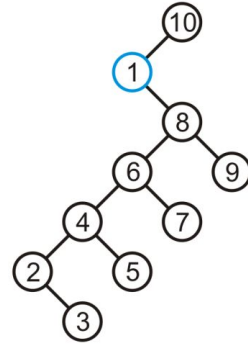
→ And yet another zig-zig rotation



Example

Splay Tree

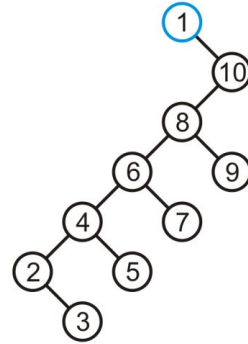
- At this point, this requires a single zig operation to bring 1 to the root



Example

Splay Tree

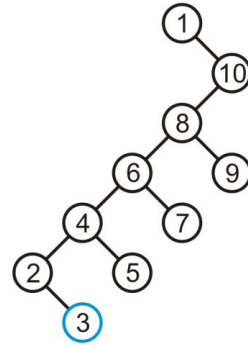
→ The height of this tree is now 6 and no longer 9



Example

Splay Tree

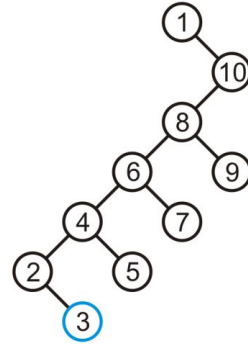
- The deepest node is now 3
- This node must be splayed to the root. **What operation do we need?**



Example

Splay Tree

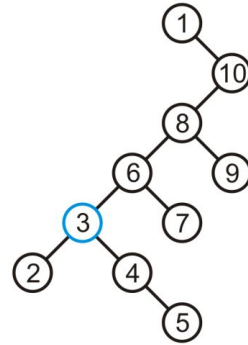
- The deepest node is now 3
- This node must be splayed to the root, start with a zig-zag operation



Example

Splay Tree

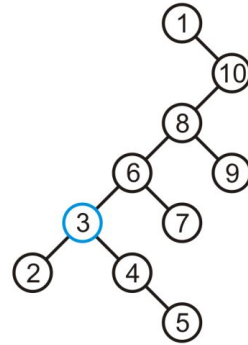
- The node 3 is rotated up
- **What do we do next?**



Example

Splay Tree

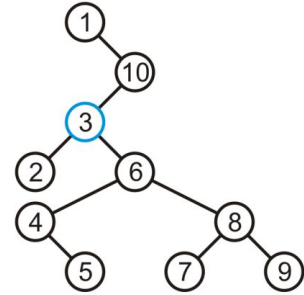
- The node 3 is rotated up
- Next we require a zig-zig operation



Example

Splay Tree

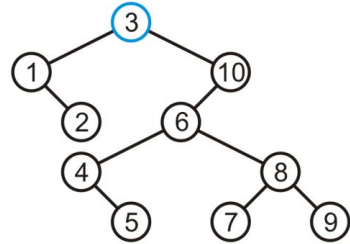
→ Finally, to bring 3 to the root, we need a zig-zag operation



Example

Splay Tree

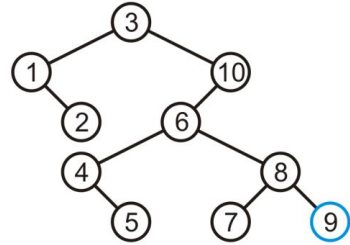
- The height of this tree is only 4
- **Which of the deepest nodes will require a zig-zig operation?**



Example

Splay Tree

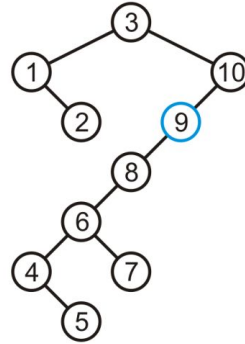
- The height of this tree is only 4
- We will remove 9 next
- The zig-zig operation will push 6 and its left subtree down



Example

Splay Tree

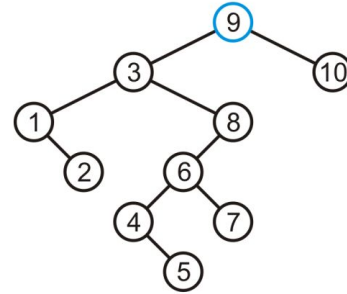
- This is closer to a linked list
- However, we're not finished
- A zig-zag operation will move 9 to the root of the tree



Example

Splay Tree

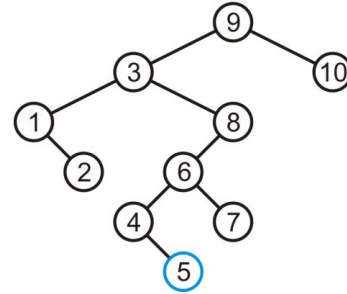
- In this case, the height of the tree is now greater than it was before: 5
- Before splaying 9, the height was 4



Example

Splay Tree

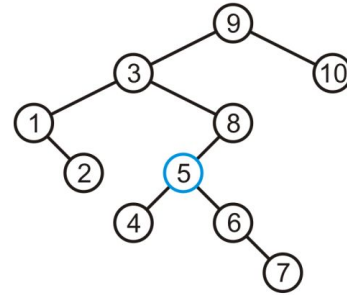
- Accessing the deepest node, 5, we must begin with a zig-zag operation



Example

Splay Tree

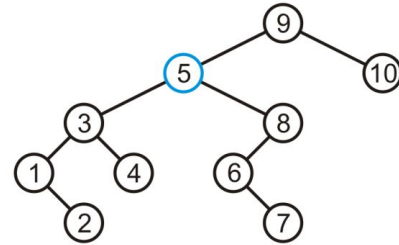
→ Next, we require a zig-zag operation to move 5 to the location of 3



Example

Splay Tree

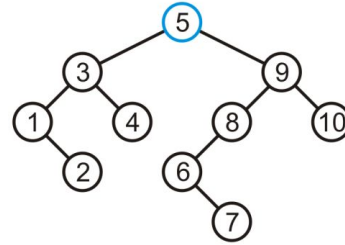
→ Finally, we require a single zig operation to move 5 to the root



Example

Splay Tree

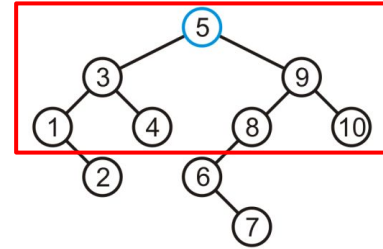
- The height of the tree is 4
- However, 7 of the nodes form a **perfect tree** at the root!



Example

Splay Tree

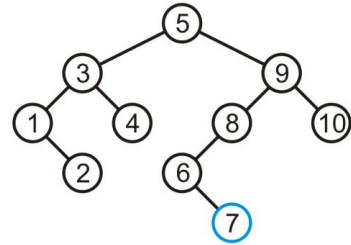
- The height of the tree is 4
- However, 7 of the nodes form a **perfect tree** at the root!



Example

Splay Tree

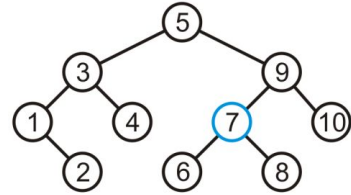
→ Next we access 7, which will require two zig-zag operations



Example

Splay Tree

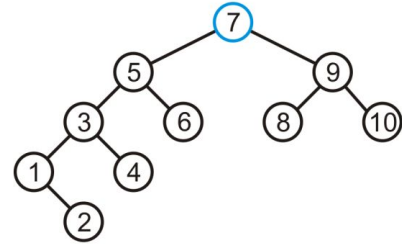
→ The first zig-zag moves it to depth 2



Example

Splay Tree

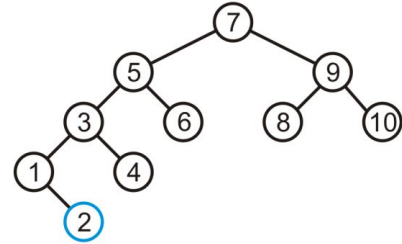
→ Next, 7 is promoted to the root through a zig-zag operation



Example

Splay Tree

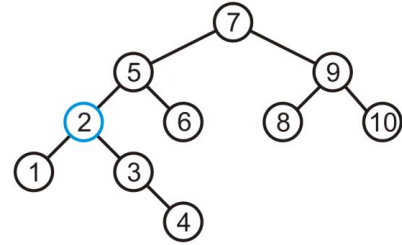
→ Finally, accessing 2, we first require a zig-zag operation



Example

Splay Tree

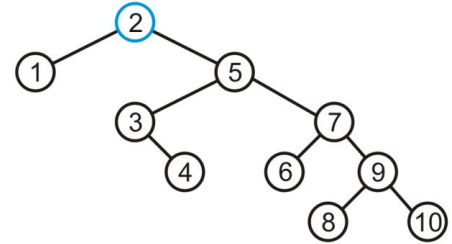
→ This now requires a zig-zig operation to promote 2 to the root



Example

Splay Tree

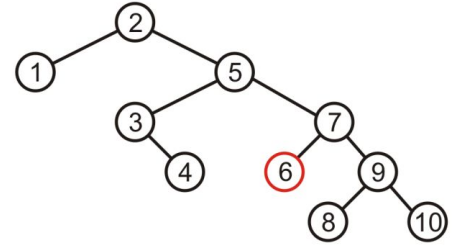
- In this case, with 2 at the root, 3-10 must be in the right subtree
- The right subtree in this case happens to be AVL balanced



Example

Splay Tree

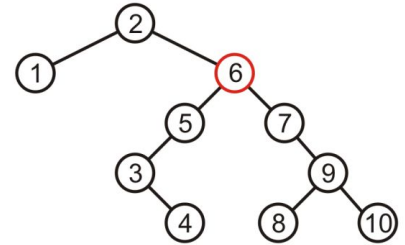
- To remove a node, for example, 6, splay it to the root
- First we require a zig-zag operation



Example

Splay Tree

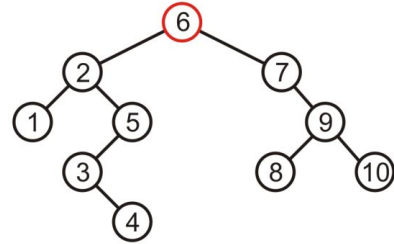
- At this point, we need a zig operation to move 6 to the root



Example

Splay Tree

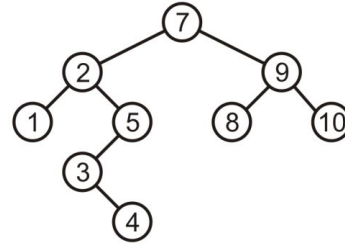
- We will now copy the minimum element from the right subtree
- The node with 7 has a single subtree, we will simply move it up



Example

Splay Tree

- Thus, we have removed 6
- The resulting tree is, once again, reasonably balanced



Performance

Runtime Analysis

- It is very difficult with small trees to demonstrate the amortized logarithmic behavior of splay trees
- The original [ACM article](#) proves the balance theorem:

The run time of performing a sequence of m operations on a splay tree with n nodes is

$$O(m(1 + \log(n)) + n \cdot \log(n))$$

- Therefore the run time for a splay tree is comparable to any balanced tree assuming at least n operations

Comparisons

Pros and Cons

Advantages:

- The amortized run times are similar to that of balanced trees such as AVL trees and red-black trees
- The implementation is easier
- No additional information (height/color) is required

Disadvantages:

- The tree structure will change with read-only operations

Summary

Splay Trees

- A binary search tree (BST)
- Splay nodes when:
 - ◆ Accessed
 - ◆ Inserted: after regular BST insertion
 - ◆ Removal options
 - Splay parent of removed node after BST removal
 - Splay node to be removed and remove node like a BST
- The height is $O(n)$ but guarantees amortized runtime of $\Theta(\log(n))$ for $\Omega(n)$ operations
- Operations are:
 - ◆ zig, zig-zag and zig-zig
- Requires no additional memory

Acknowledgement: These slides are adapted from Professor Douglas Wilhelm Harder's ECE250 course at University of Waterloo.